

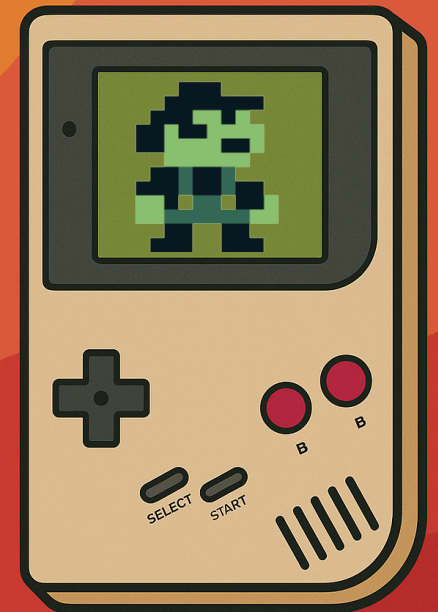


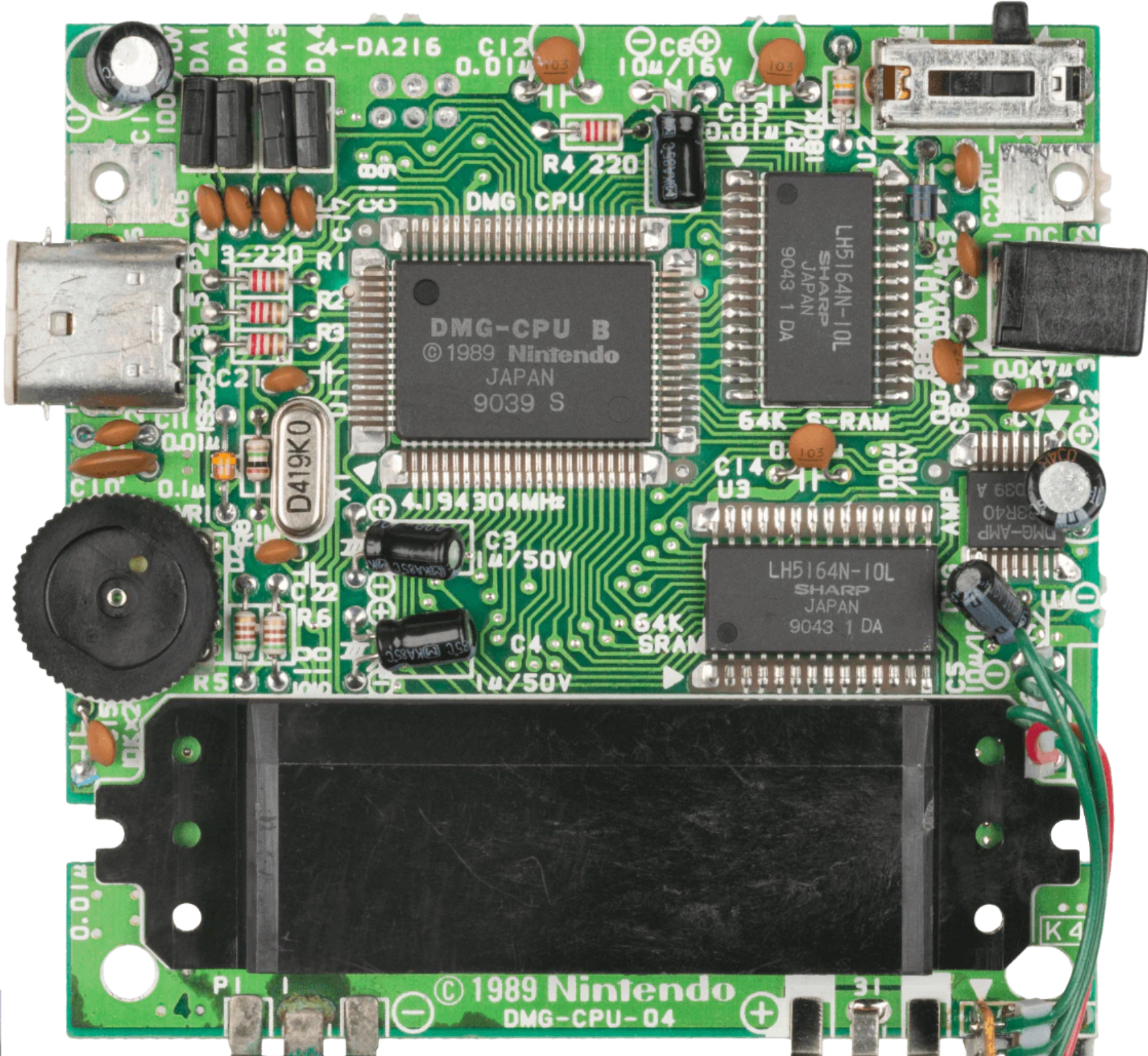
Arquitectura de ComputadorasTM

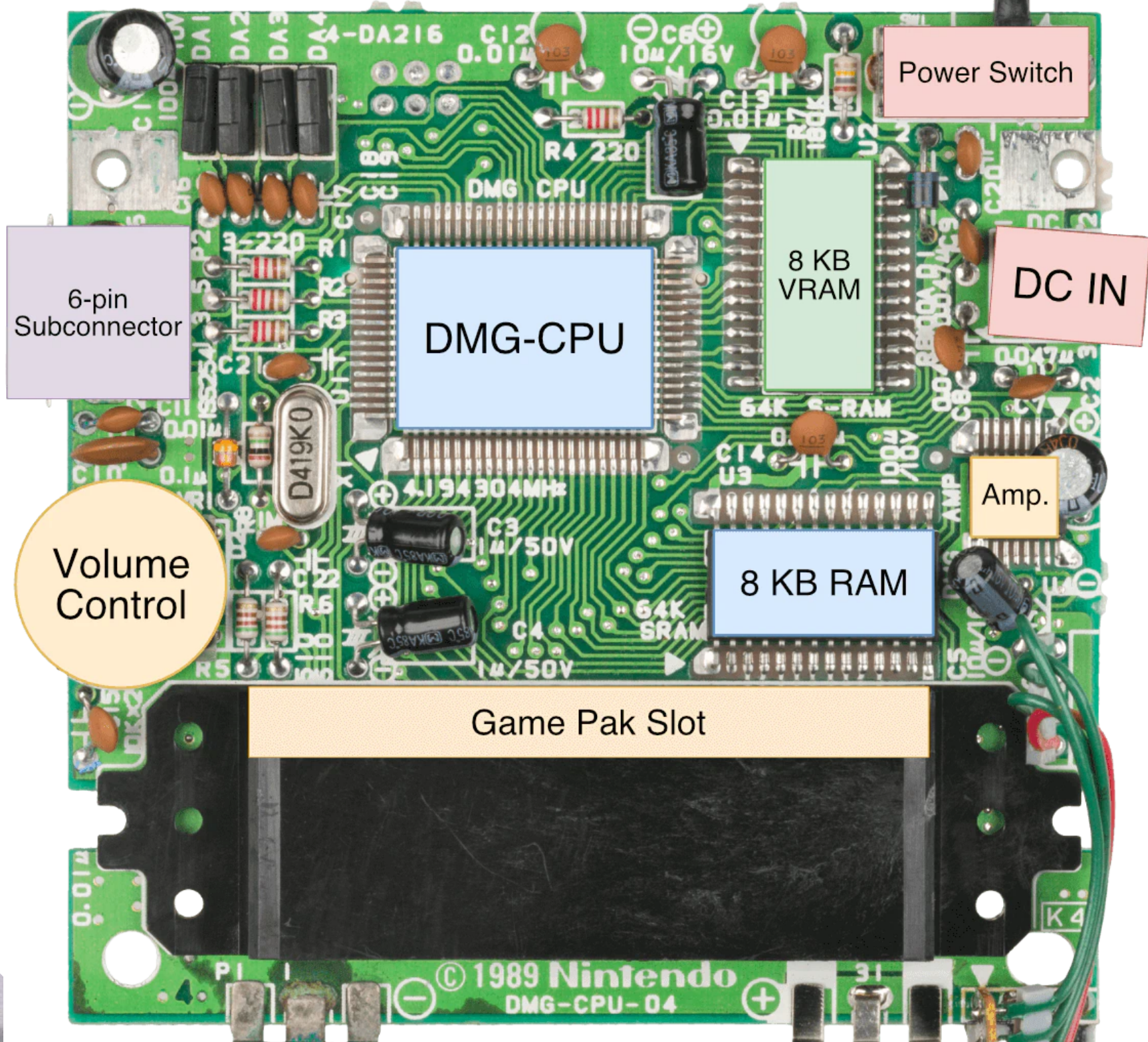
Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta
Erik Borgnia

Historia - ¿Game Boy?

- Consola portátil desarrollada y fabricada por Nintendo.
- Salió a la venta por primera vez en Abril de 1989, en Japón.
- Llamada internamente como DMG, siglas de Dot Matrix Game, por su pantalla de matriz de puntos.
- Tiene pantalla sin colores no retroiluminada, con un tinte verde que representa 4 tonos de grises.
- Tiene un total de 8KiB de VRAM, y 8KiB de RAM.
- Vendió casi 100 millones de unidades
- Su juego más vendido fue Super Mario Land (~30M)





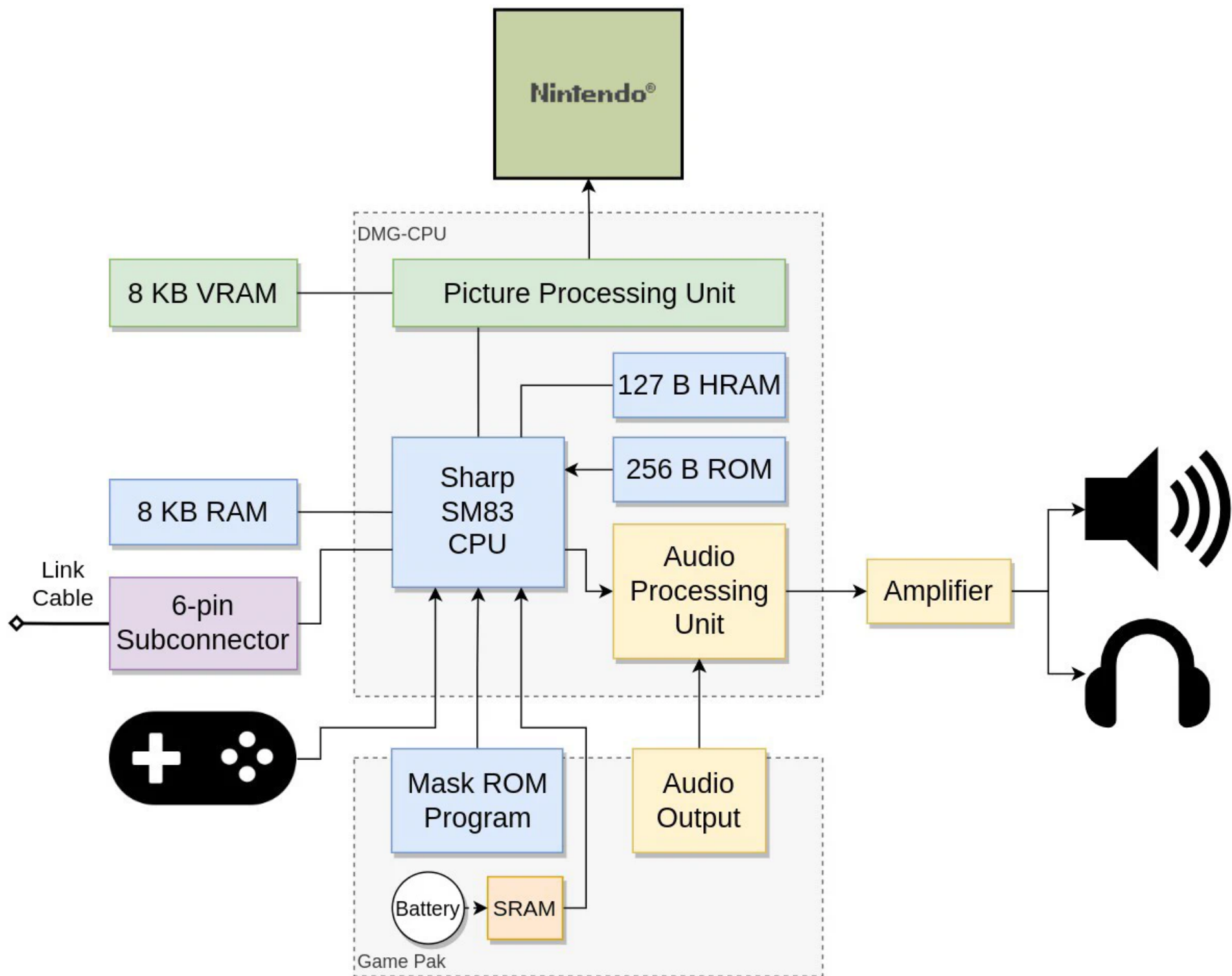


DMG-CPU... O mejor dicho, SoC

La DMG-CPU en realidad no es una CPU, es un SoC (System on a Chip).

¡Es un circuito integrado que contiene múltiples componentes!

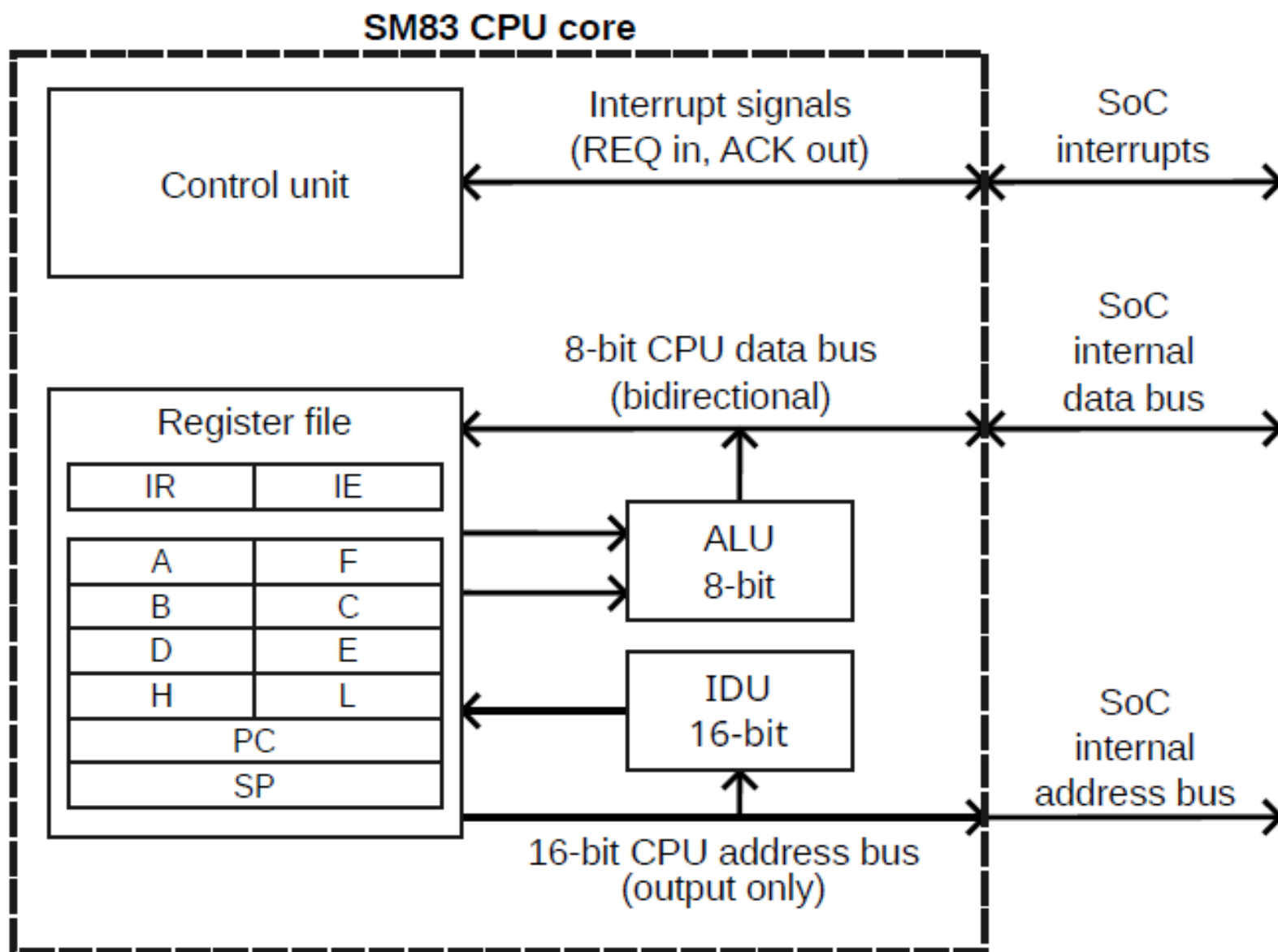




Características generales del CPU Sharp SM83

- Bus de datos de 8 bits
- Bus de direcciones de 16 bits (65536 celdas direccionables = 64KiB)
- Contiene una ALU de 8 bits, y una IDU (Inc. Dec. Unit) de 16 bits
- 7 registros de 8 bits de uso general (**a**-acumulador, **b**, **c**, **d**, **e**, **h**, **l**)
- Provee algunas operaciones de 16 bits
- 3 registros de 16 bits de uso general (**bc**, **de**, **hl**)
- Provee la posibilidad de uso de interrupciones (registro IE)
- Búsqueda y Ejecución en paralelo (simil pipeline)
- **TODO** está mapeado de forma directa (direccionable a 16 bits)

Características generales del CPU Sharp SM83



Lenguaje ensamblador del SM83

El procesador tiene un set de instrucciones similar al Intel 8080, lo cual incluye, pero no está limitado a, las siguientes instrucciones:

- ♦ **ld OpA, OpB** - Equivalente al MOV de la MV. Se puede hacer de registro a registro (o [hl]), de inmediato a registro (o [hl]), o desde/hacia el registro **a** hacia/desde una indirección, a través de un registro de 16 o un inmediato.
- ♦ **add OpA** - Similar al ADD de la MV, pero con el registro **a** como registro de salida. OpA puede ser un registro, un inmediato, o [hl].
- ♦ **adc/sub/sbc/and/xor/or/cp OpA** - Mismo concepto que add.
- ♦ **swap OpA** - Intercambia nibbles de lugar (0xAB -> 0xBA). OpA puede ser un registro o [hl].

Lenguaje ensamblador del SM83

- ♦ **jp CC, NN** - Equivalente a los distintos saltos de la MV. CC indica la bandera (Z, C, NZ, NC), y NN es un inmediato.
- ♦ **jp OpA** - Salto incondicional, OpA es un inmediato o hl.
- ♦ **call CC, NN** - Salto a subrutina. CC indica la bandera (Z, C, NZ, NC, o podría no estar), NN es un inmediato.
- ♦ **ret CC** - Retorno de subrutina. CC indica la bandera (Z, C, NZ, NC, o podría no estar).
- ♦ **push Reg16** - Almacena en la pila, Reg16 es AF, BC, DE o HL.
- ♦ **pop Reg16** - Extrae de la pila, Reg16 es AF, BC, DE o HL.
- ♦ **entre varios otros...**

Distribución de la memoria

Todo, incluyendo los periféricos, está mapeado de forma directa:

- ◆ ROM del Cartucho (0x0000 - 0x7FFF)
- ◆ VRAM (0x8000 - 0x9FFF y 0xFE00 - 0xFE9F) *
- ◆ RAM (0xC000 - 0xDFFF)
- ◆ Joypad, Sonido, Audio, Serial (0xFF00 - 0xFF7F) **
- ◆ HRAM (0xFF80 - 0xFFFE)
- ◆ Registro IE (0xFFFF)
- ◆ Etc...

* Incluyendo OAM, que se usa para mostrar los objetos en pantalla.

** Entre otras cosas que conforman la comunicación de la consola con los periféricos.

Distribución de la memoria (0x0000-0x7FFF)

Dir. de Memoria	¿Qué hay?	Tamaño
0x0000 - 0x00FF	Vectores RST e Interrupciones	
0x0100 - 0x014F	Header de Cartucho	
0x0150 - 0x3FFF	ROM de Cartucho	Banco 0 16 KiB
0x4000 - 0x7FFF	ROM de Cartucho (banco intercambiable)	Banco 1-N 16 KiB

Distribución de la memoria (0x8000-0xFFFF)

Dir. de Memoria	¿Qué hay?		Tamaño
0x8000 - 0x97FF	Datos de Tiles (objetos y fondo)	VRAM	8 KiB
0x9800 - 0x9BFF	Mapa de Tiles 1 (fondo 1)		
0x9C00 - 0x9FFF	Mapa de Tiles 2 (fondo 2)		
0xA000 - 0xBFFF	RAM de Cartucho (si existe)	RAM Ext.	8 KiB
0xC000 - 0xDFFF	RAM Interna	RAM	8 KiB
0xE000 - 0xFDFF	ECHO RAM (NO USABLE)	ECHO	7,5 KiB
0xFE00 - 0xFE9F	Memoria de Atributos de Objetos	OAM	160 Bytes
0xFEA0 - 0xFEFF	Memoria NO USABLE	-	96 Bytes
0xFF00 - 0xFF7F	Registros de I/O del Hardware	I/O	128 Bytes
0xFF80 - 0xFFFF	RAM de Alta Velocidad	HRAM	127 Bytes
0xFFFF	Bandera de Interrupción	IE	1 Byte

Joypad

La lectura del joypad se realiza a través del byte en [0xFF00], el cual tiene la siguiente estructura:

Bit Nro.	7	6	5	4	3	2	1	0
	No usados		Selector de Botones	Selector de D-Pad	Start / Abajo	Select / Arriba	B / Izquierda	A / Derecha

El uso del mismo es mediante una escritura sobre los selectores en los bits 4 y 5, los cuales actúan (de forma física en la consola) como “puentes” hacia los interruptores de la consola. Nótese que estos selectores, así como las señales de que algún botón está presionado, funcionan como “pull-down”, es decir que un 1 significa desactivado y un 0 significa activado!



Por ejemplo, si estuvieran apretados Derecha, Arriba, y A:

Bit Nro.	7	6	5	4	3	2	1	0
	No usados		0	1	1	1	1	0

Bit Nro.	7	6	5	4	3	2	1	0
	No usados		1	0	1	0	1	0

Botones

D-Pad

NOTA: Si ambos selectores están en 1 (no seleccionados), en el nibble inferior se lee 0xF

Joypad - Código de ejemplo

Un código para la lectura del joypad podría ser el siguiente:

```
ld a, $20 ;Selecciono D-Pad
ld [$FF00], a
ld a, [$FF00]
and a, $0F ;En el nibble bajo quedan las direcciones seleccionadas
swap a ;Invierto nibbles
ld b, a
ld a, $10 ;Selecciono botones
ld [$FF00], a
ld a, [$FF00]
and a, $0F ;En el nibble bajo quedan los botones seleccionados
or a, b ;En a queda todo el joypad, pero en sentido PULL-DOWN!
xor a, $FF ;Invierto, para que quede en 1 lo presionado
```

Graficos... ¿cómo funcionan?

La Game Boy tiene una pantalla de **160x144** píxeles

Hagamos unos cálculos:

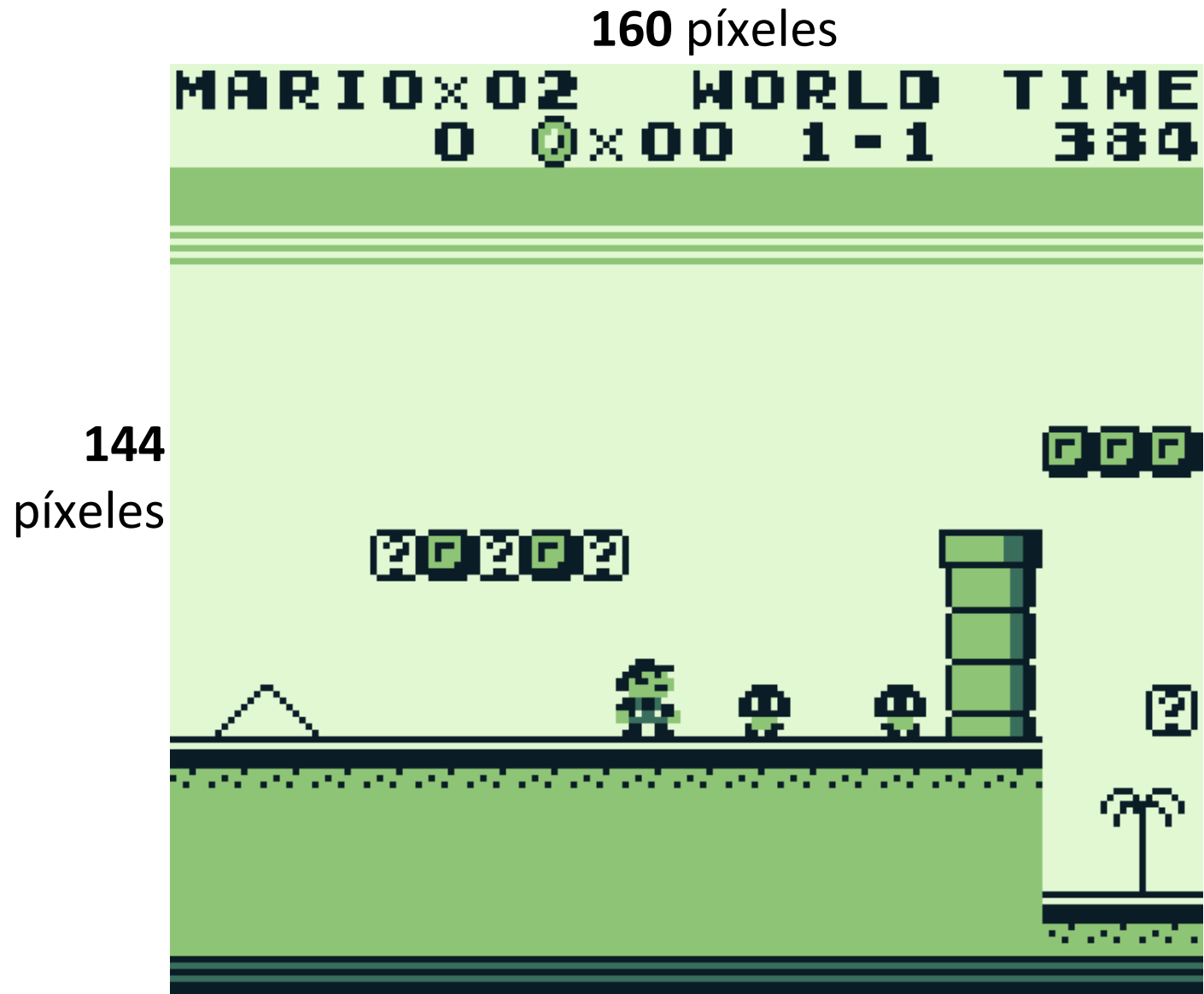
160 * **144** = **23040** píxeles

Cada píxel tiene 4 tonos

23040 * 2 bits = 46080 bits
= **5760 Bytes**

Tenemos **8192 Bytes** de VRAM, por lo que nos alcanza... verdad?

A menos que...



Graficos... ¿cómo funcionan?

¡160x144 es sólo lo visible!
Para que los desplazamientos
puedan ser suaves, se
requiere un fondo más
grande.

$256 * 256 = 64 \text{ Ki}$ píxeles

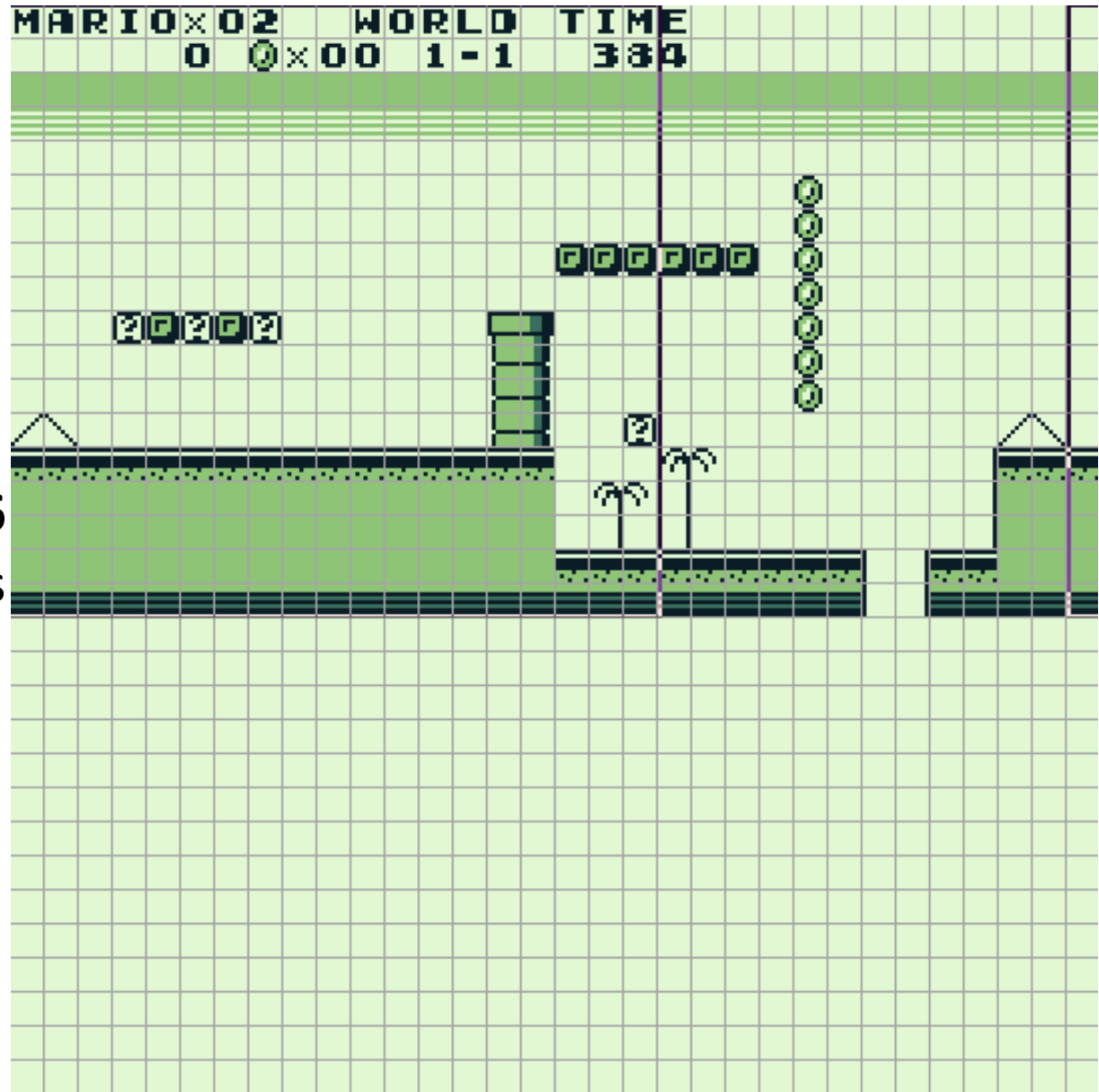
$64 \text{ Ki} * 2 \text{ bits} = 128 \text{ Kib}$
 $= 16 \text{ KiB!}$

Esto es el doble de lo que
disponemos.

¿Cuál es la solución?
¡TILES!

256 píxeles

256
píxeles

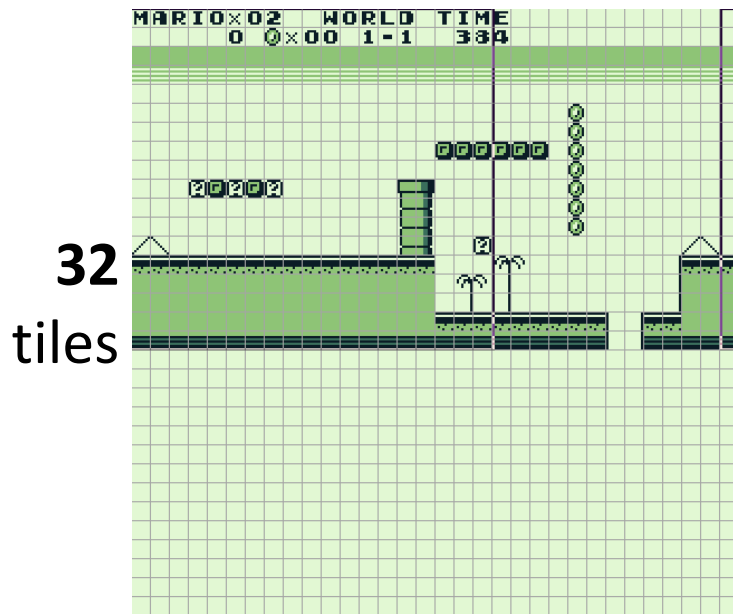


Tiles y mapas de tiles

Un tile, o mosaico, es una sección de 8x8 píxeles, con las cuales se arman los fondos, usando mapas de tiles.



$8 * 8 = 64$ píxeles
 $64 * 2 \text{ bits} = 16 \text{ Bytes}$
¡POR CADA TILE!



32
tiles

32 tiles

32 tiles * 32 tiles = 1024 tiles (por mapa)

Si tuviéramos 256 tiles distintos para usar en el fondo, los podríamos identificar con números de 8 bits (1 Byte) por tile, lo que hace que todas las referencias del fondo ocupen 1 KiB.

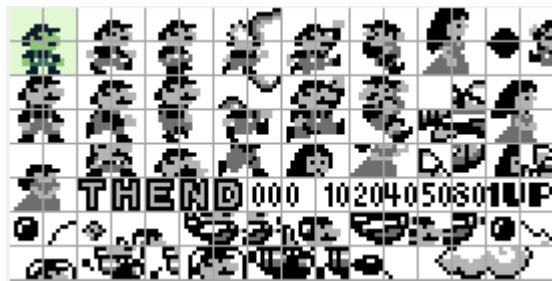
... nos alcanza?

$256 \text{ tiles} * 16 \text{ Bytes} = 4 \text{ KiB}$

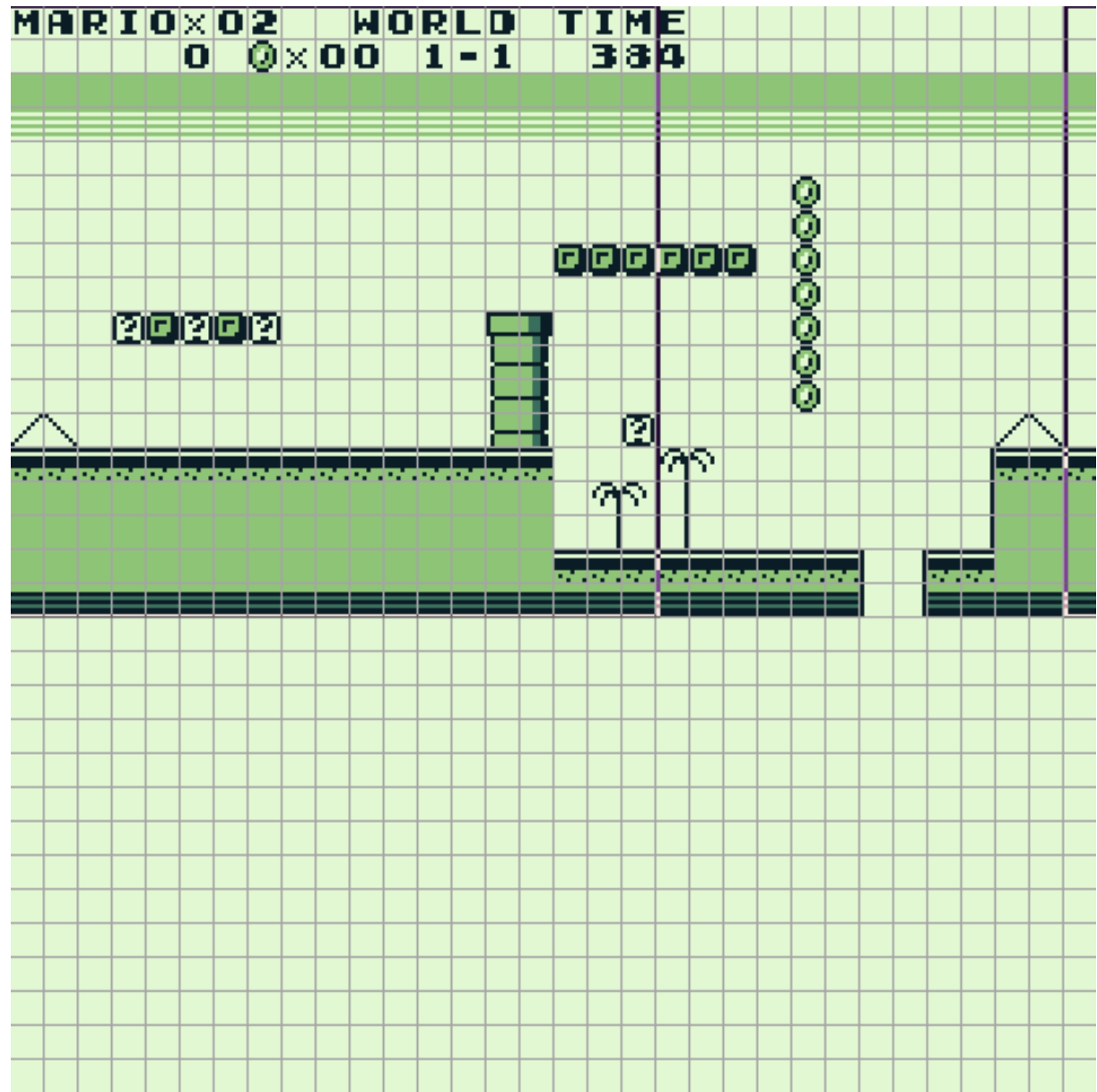
Esto es la mitad de nuestra VRAM. Si tomamos además el 1 KiB del fondo, nos quedan 3 KiB extra. Pero nos falta algo...

Objetos...

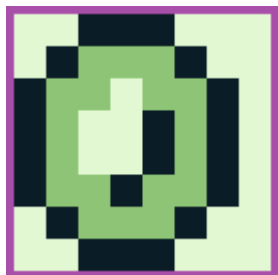
Los mapas de tiles son útiles para los fondos, que son elementos fijos del juego, pero para los elementos que se mueven se usan objetos, ¡los cuales también usan tiles! Pero si usamos 256, no nos alcanza la VRAM (serían 8KiB sin el mapa de tiles). Entonces podemos usar 128, y compartir 128 con los fondos, así ambos tienen 256!



¿Dónde está Mario? ¿Y los Chibibo?



¿Cómo se ven los tiles y los mapas de tiles en memoria?



Los tiles usan 2 bits por pixel, con 00 representando el color más claro, hasta 11 que representa el más oscuro. En tiles de objetos, el 00 representa transparencia (es decir, hay sólo 3 colores).

```
00 00 11 11 11 00 00 00
00 11 01 01 01 11 00 00
11 01 01 00 01 01 11 00
11 01 00 00 11 01 11 00
11 01 00 00 11 01 11 00
11 01 01 11 01 01 11 00
00 11 01 01 01 11 00 00
00 00 11 11 11 00 00 00
```

Aunque en la realidad, estos datos se almacenan bit a bit en Little Endian, por lo que cada fila de 8 píxeles se expresa como 4 hexadecimales:

00 11 01 01 01 11 00 00 (segunda fila de pixeles)

0111 1100 0100 0100

0x7C

0x44

Quedándonos en la memoria (en hexadecimal):

38 38 7C 44 (filas 1 y 2)

EE 82 CE 8A (filas 3 y 4)

CE 8A FE 92 (filas 5 y 6)

7C 44 38 38 (filas 7 y 8)

Para un total de
16 Bytes.

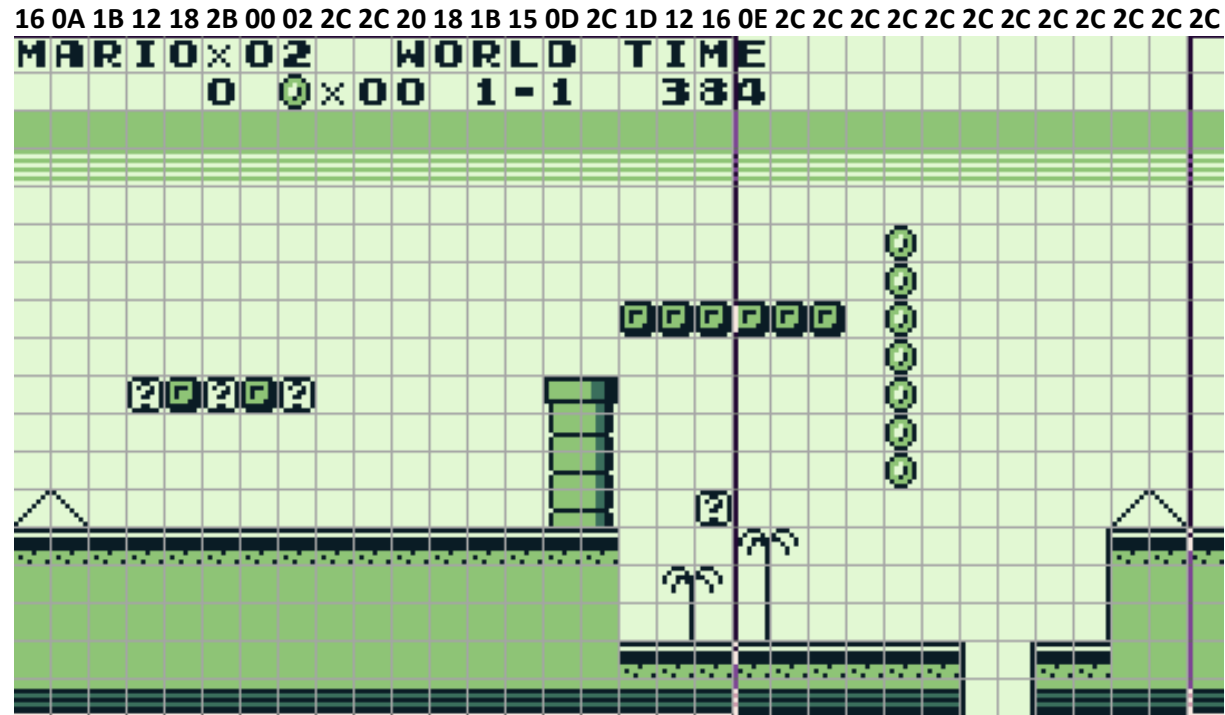
Técnicamente acá el color hace referencia a la paleta, la cual generalmente se usa de esta forma!

Tarea para casa: verificar el resto...

¿Cómo se ven los tiles y los mapas de tiles en memoria?

Para los mapas de tiles es muchísimo más simple. Como hay 256 tiles, cada uno es representado por 1 Byte. En la memoria está simplemente cada índice de tile, de forma consecutiva, por filas, en este caso veríamos, en hexadecimal:

16 0A 1B 12 18 2B 00 02 (8 tiles)
2C 2C 20 18 1B 15 0D 2C (16 tiles)
1D 12 16 0E 2C 2C 2C 2C (24 tiles)
2C 2C 2C 2C 2C 2C 2C 2C (32 tiles)
 (y hasta acá la primer fila de 32, se
 podrán hacer una idea del resto)



Resumiendo

Tenemos un total de 384 tiles, 128 para objetos, 128 para fondos, y 128 compartidos. $384 \text{ tiles} * 16 \text{ Bytes} = 6 \text{ KiB}$. Sumando al mapa de tiles, tenemos 7 KiB. Con el KiB que nos sobra, podemos hacer un segundo mapa de tiles, al cual se le llama ventana.



Tiles objeto
0x00-0x7F

En común
0x80-0xFF

Tiles fondo
0x00-0x7F

Este trabajo de interpretar y mostrar los gráficos lo hace la PPU, la cual da una tasa de refresco aproximada de 60 Hz.

Recordando cómo se direcciona la memoria...

Dir. de Memoria	¿Qué hay?	Tamaño
0x8000 - 0x97FF	Datos de Tiles (objetos y fondo)	VRAM 8 KiB
0x9800 - 0x9BFF	Mapa de Tiles 1 (fondo 1)	
0x9C00 - 0x9FFF	Mapa de Tiles 2 (fondo 2)	

¿Y la música? ¿Comunicación serial? ¿Interrupciones?

- Cada área de la consola tiene sus particularidades. El objetivo de esta clase no es que salgan siendo expertos en programación para Game Boy, si no darles un vistazo al funcionamiento en general de su arquitectura.
- Aquí podemos ver como con un procesador muy sencillo, pero con un poco de imaginación, se pueden hacer cosas muy interesantes, como manejo de gráficos, sonido, etc.
- Si les interesa meterse más a fondo, ¡adelante! Es todo un mundo por descubrir. Eso sí, conviene tener a mano algo para el dolor de cabeza. En las referencias tienen todo lo necesario.

Referencias

- Game Boy/Color Architecture: A Practical Analysis, Rodrigo Copetti, <https://www.copetti.org/>
- Game Boy: Complete Technical Reference, <https://gekkio.fi/>
- Game Boy Pandocs, <https://gbdev.io/>



Arquitectura de ComputadorasTM

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta
Erik Borgnia